

Отчёт по лабораторной работе 7

Архитектура компьютера

Ёров Муроджон НКАбд-03-25

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
2.1	Реализация переходов в NASM	6
2.2	Изучение структуры файлы листинга	13
2.3	Задание для самостоятельной работы	15
3	Выводы	20

Список иллюстраций

2.1	Программа в файле lab7-1.asm	7
2.2	Запуск программы lab7-1.asm	7
2.3	Программа в файле lab7-1.asm	9
2.4	Запуск программы lab7-1.asm	9
2.5	Программа в файле lab7-1.asm	10
2.6	Запуск программы lab7-1.asm	11
2.7	Программа в файле lab7-2.asm	12
2.8	Запуск программы lab7-2.asm	12
2.9	Файл листинга lab7-2	13
2.10	Ошибка трансляции lab7-2	14
2.11	Файл листинга с ошибкой lab7-2	15
2.12	Программа в файле task7-1.asm	16
2.13	Запуск программы task7-1.asm	16
2.14	Программа в файле task7-2.asm	18
2.15	Запуск программы task7-2.asm	19

Список таблиц

1 Цель работы

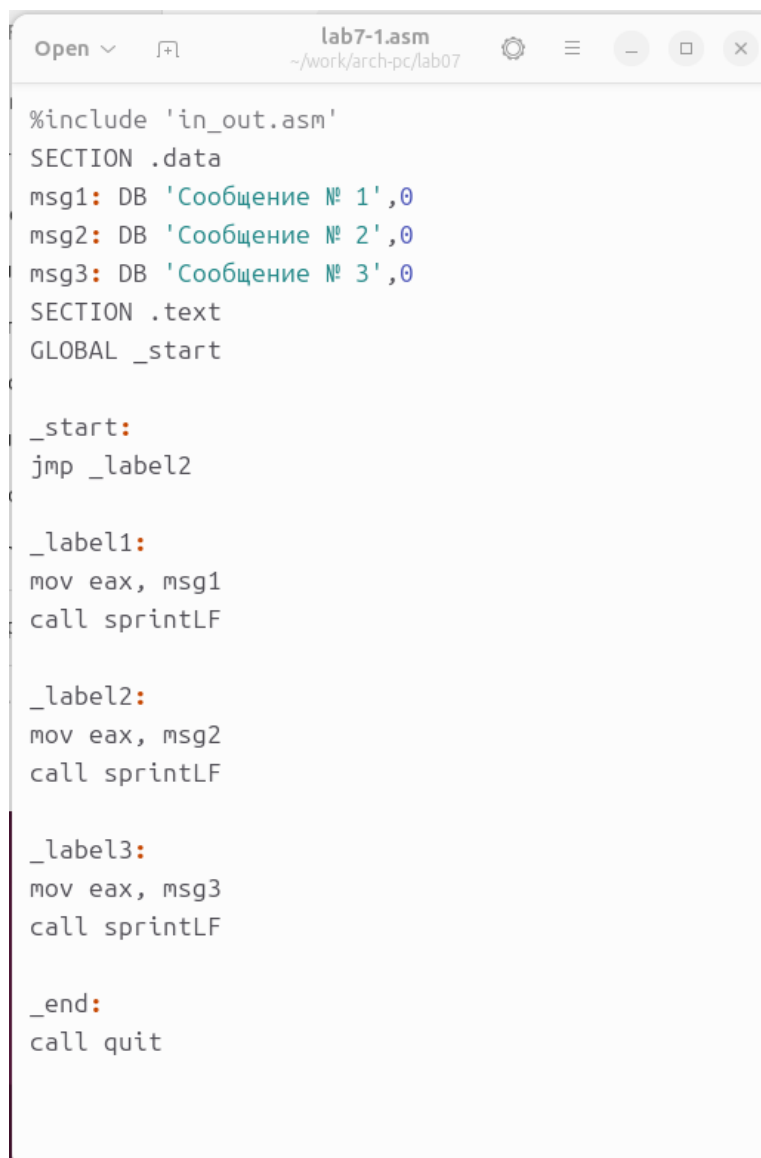
Целью работы является изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга.

2 Выполнение лабораторной работы

2.1 Реализация переходов в NASM

Создал каталог для программ лабораторной работы № 7 и файл lab7-1.asm

Инструкция `jmp` в NASM используется для реализации безусловных переходов. Рассмотрим пример программы с использованием инструкции `jmp`. Написал в файл lab7-1.asm текст программы из листинга 7.1.



```
lab7-1.asm
~/work/arch-pc/lab07

%include 'in_out.asm'
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start

_start:
jmp _label2

_label1:
mov eax, msg1
call sprintLF

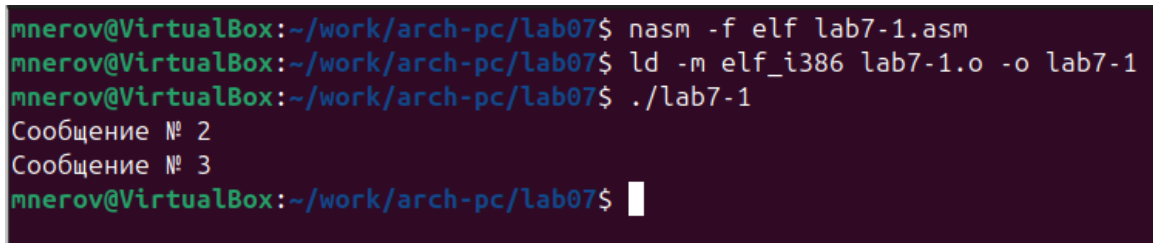
_label2:
mov eax, msg2
call sprintLF

_label3:
mov eax, msg3
call sprintLF

_end:
call quit
```

Рисунок 2.1: Программа в файле lab7-1.asm

Создал исполняемый файл и запустил его.



```
mnerov@VirtualBox:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
mnerov@VirtualBox:~/work/arch-pc/lab07$ ld -m elf_i386 lab7-1.o -o lab7-1
mnerov@VirtualBox:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 3
mnerov@VirtualBox:~/work/arch-pc/lab07$
```

Рисунок 2.2: Запуск программы lab7-1.asm

Инструкция `jmp` позволяет осуществлять переходы не только вперед но и назад. Изменим программу таким образом, чтобы она выводила сначала „Сообщение № 2“, потом „Сообщение № 1“ и завершала работу. Для этого в текст программы после вывода сообщения № 2 добавим инструкцию `jmp` с меткой `_label1` (т.е. переход к инструкциям вывода сообщения № 1) и после вывода сообщения № 1 добавим инструкцию `jmp` с меткой `_end` (т.е. переход к инструкции `call quit`).

Изменил текст программы в соответствии с листингом 7.2.



```
Open ▾  lab7-1.asm
~/.work/arch-pc/lab07

%include 'in_out.asm'
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start

_start:
jmp _label2

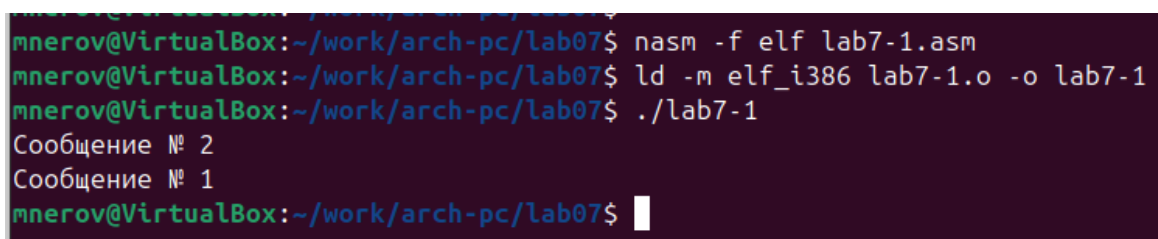
_label1:
mov eax, msg1
call sprintLF
jmp _end

_label2:
mov eax, msg2
call sprintLF
jmp _label1

_label3:
mov eax, msg3
call sprintLF

_end:
call quit
```

Рисунок 2.3: Программа в файле lab7-1.asm



```
mnerov@VirtualBox:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
mnerov@VirtualBox:~/work/arch-pc/lab07$ ld -m elf_i386 lab7-1.o -o lab7-1
mnerov@VirtualBox:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 1
mnerov@VirtualBox:~/work/arch-pc/lab07$
```

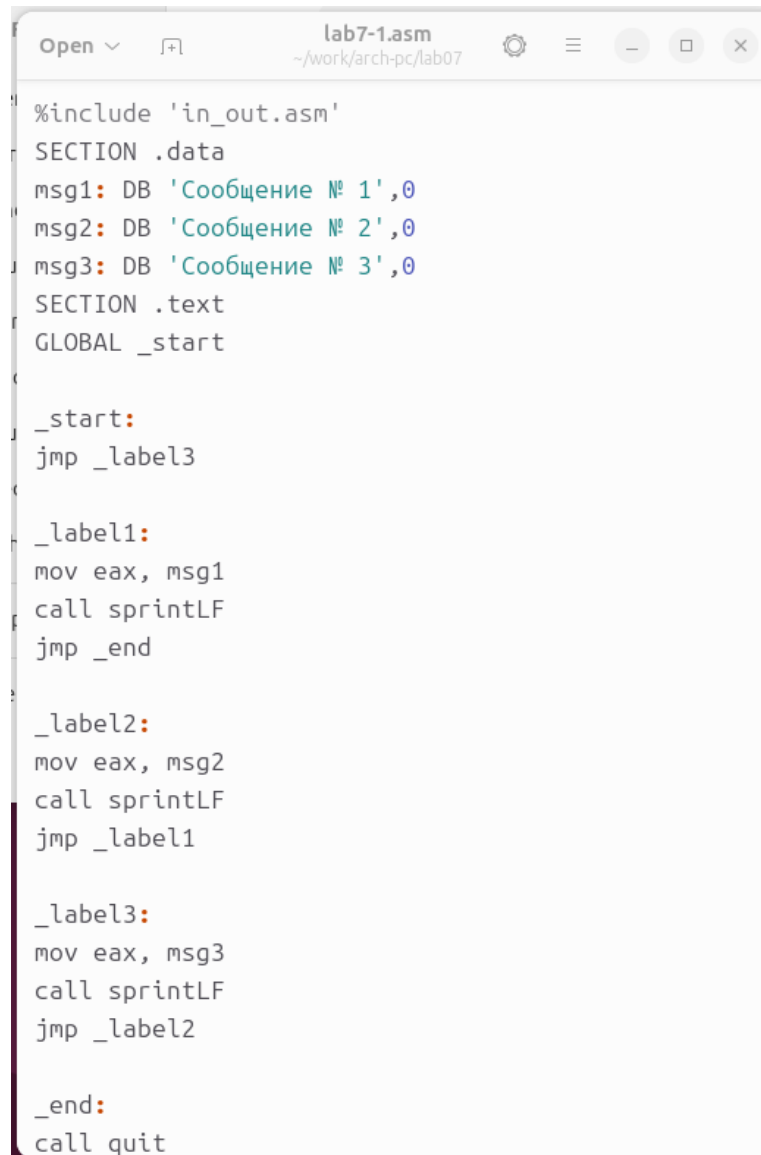
Рисунок 2.4: Запуск программы lab7-1.asm

Изменил текст программы, изменив инструкции `jmp`, чтобы вывод программы был следующим:

Сообщение № 3

Сообщение № 2

Сообщение № 1



```
Open ▾ [F+] lab7-1.asm
~/work/arch-pc/lab07

%include 'in_out.asm'
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start

_start:
jmp _label3

_label1:
mov eax, msg1
call sprintLF
jmp _end

_label2:
mov eax, msg2
call sprintLF
jmp _label1

_label3:
mov eax, msg3
call sprintLF
jmp _label2

_end:
call quit
```

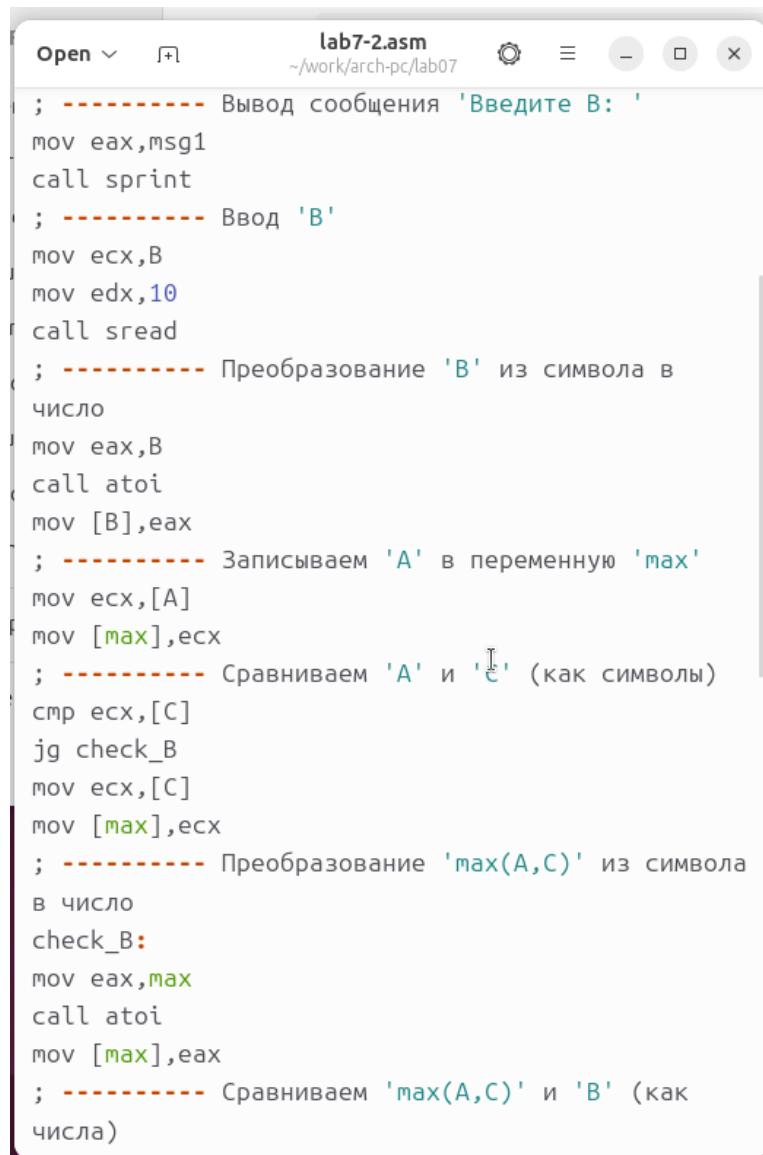
Рисунок 2.5: Программа в файле lab7-1.asm

```
mnerov@VirtualBox:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
mnerov@VirtualBox:~/work/arch-pc/lab07$ ld -m elf_i386 lab7-1.o -o lab7-1
mnerov@VirtualBox:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 3
Сообщение № 2
Сообщение № 1
mnerov@VirtualBox:~/work/arch-pc/lab07$
```

Рисунок 2.6: Запуск программы lab7-1.asm

Использование инструкции `jmp` приводит к переходу в любом случае. Однако, часто при написании программ необходимо использовать условные переходы, т.е. переход должен происходить если выполнено какое-либо условие. В качестве примера рассмотрим программу, которая определяет и выводит на экран наибольшую из 3 целочисленных переменных: А, В и С. Значения для А и С задаются в программе, значение В вводится с клавиатуры.

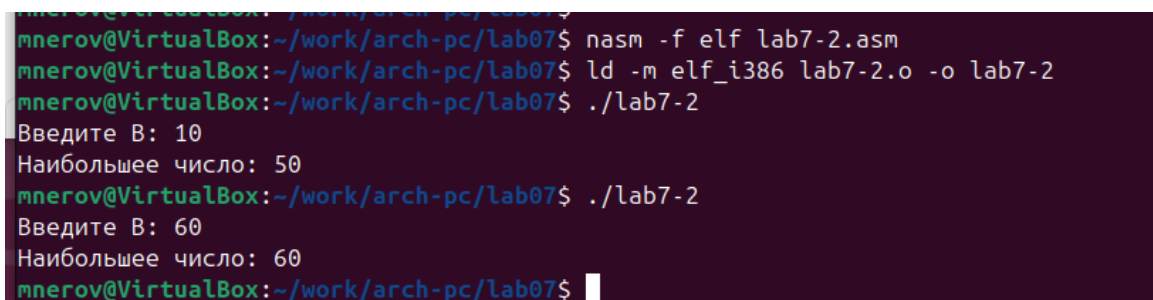
Создал исполняемый файл и проверил его работу для разных значений В.



```
Open ▾ [icon] lab7-2.asm
~/work/arch-pc/lab07

; ----- Вывод сообщения 'Введите B: '
mov eax,msg1
call sprint
; ----- Ввод 'B'
mov ecx,B
mov edx,10
call sread
; ----- Преобразование 'B' из символа в
число
mov eax,B
call atoi
mov [B],eax
; ----- Записываем 'A' в переменную 'max'
mov ecx,[A]
mov [max],ecx
; ----- Сравниваем 'A' и 'C' (как символы)
cmp ecx,[C]
jg check_B
mov ecx,[C]
mov [max],ecx
; ----- Преобразование 'max(A,C)' из символа
в число
check_B:
mov eax,max
call atoi
mov [max],eax
; ----- Сравниваем 'max(A,C)' и 'B' (как
числа)
```

Рисунок 2.7: Программа в файле lab7-2.asm



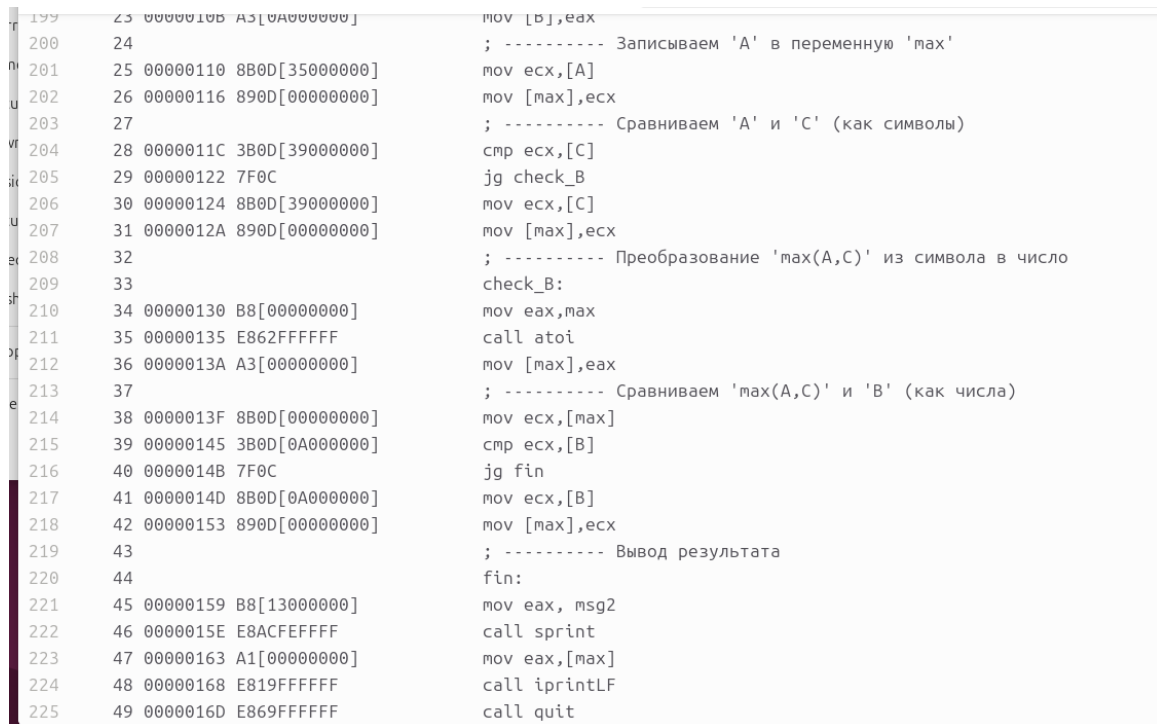
```
mnerov@VirtualBox:~/work/arch-pc/lab07$ nasm -f elf lab7-2.asm
mnerov@VirtualBox:~/work/arch-pc/lab07$ ld -m elf_i386 lab7-2.o -o lab7-2
mnerov@VirtualBox:~/work/arch-pc/lab07$ ./lab7-2
Введите B: 10
Наибольшее число: 50
mnerov@VirtualBox:~/work/arch-pc/lab07$ ./lab7-2
Введите B: 60
Наибольшее число: 60
mnerov@VirtualBox:~/work/arch-pc/lab07$
```

Рисунок 2.8: Запуск программы lab7-2.asm

2.2 Изучение структуры файлы листинга

Обычно `nasm` создаёт в результате ассемблирования только объектный файл. Получить файл листинга можно, указав ключ `-l` и задав имя файла листинга в командной строке.

Создал файл листинга для программы из файла `lab7-2.asm`



```
199 23 0000010B A3[0A000000]    mov [B],eax
200 24                          ; ----- Записываем 'A' в переменную 'max'
201 25 00000110 8B0D[35000000]    mov ecx,[A]
202 26 00000116 890D[00000000]    mov [max],ecx
203 27                          ; ----- Сравниваем 'A' и 'C' (как символы)
204 28 0000011C 3B0D[39000000]    cmp ecx,[C]
205 29 00000122 7F0C              jg check_B
206 30 00000124 8B0D[39000000]    mov ecx,[C]
207 31 0000012A 890D[00000000]    mov [max],ecx
208 32                          ; ----- Преобразование 'max(A,C)' из символа в число
209 33                          check_B:
210 34 00000130 B8[00000000]    mov eax,max
211 35 00000135 E862FFFFFF        call atoi
212 36 0000013A A3[00000000]    mov [max],eax
213 37                          ; ----- Сравниваем 'max(A,C)' и 'B' (как числа)
214 38 0000013F 8B0D[00000000]    mov ecx,[max]
215 39 00000145 3B0D[0A000000]    cmp ecx,[B]
216 40 0000014B 7F0C              jg fin
217 41 0000014D 8B0D[0A000000]    mov ecx,[B]
218 42 00000153 890D[00000000]    mov [max],ecx
219 43                          ; ----- Вывод результата
220 44                          fin:
221 45 00000159 B8[13000000]    mov eax,msg2
222 46 0000015E E8ACFFFFFF        call sprint
223 47 00000163 A1[00000000]    mov eax,[max]
224 48 00000168 E819FFFFFF        call iprintLF
225 49 0000016D E869FFFFFF        call quit
```

Рисунок 2.9: Файл листинга `lab7-2`

Внимательно ознакомился с его форматом и содержимым. Подробно объясню содержимое трёх строк файла листинга по выбору.

строка 203

- 28 - номер строки в подпрограмме
- 0000011C - адрес
- 3B0D[39000000] - машинный код

- `cmp esx,[C]` - код программы - сравнивает регистр `esx` и переменную `C`

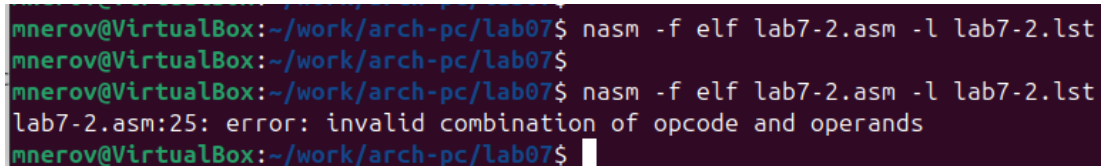
строка 204

- 29 - номер строки в подпрограмме
- 00000122 - адрес
- 7F0C - машинный код
- `jg check_V` - код программы - если `>`, то переход к метке `check_V`

строка 205

- 30 - номер строки в подпрограмме
- 00000124 - адрес
- 8B0D[39000000] - машинный код
- `mov esx,[C]` - код программы - перекладывает в регистр `esx` значение переменной `C`

Открыл файл с программой `lab7-2.asm` и в инструкции с двумя операндами удалил один операнд. Выполнил трансляцию с получением файла листинга.



```
mnerov@VirtualBox:~/work/arch-pc/lab07$ nasm -f elf lab7-2.asm -l lab7-2.lst
mnerov@VirtualBox:~/work/arch-pc/lab07$
mnerov@VirtualBox:~/work/arch-pc/lab07$ nasm -f elf lab7-2.asm -l lab7-2.lst
lab7-2.asm:25: error: invalid combination of opcode and operands
mnerov@VirtualBox:~/work/arch-pc/lab07$
```

Рисунок 2.10: Ошибка трансляции `lab7-2`

```

195 19 000000FC E842FFFFFF      call sread
196 20                          ; ----- Преобразование 'B' из символа в число
197 21 00000101 B8[0A000000]    mov eax,B
198 22 00000106 E891FFFFFF      call atoi
199 23 0000010B A3[0A000000]    mov [B],eax
200 24                          ; ----- Записываем 'A' в переменную 'max'
201 25                          mov ecx,
202 25 *****                  error: invalid combination of opcode and operands
203 26 00000110 890D[00000000]    mov [max],ecx
204 27                          ; ----- Сравниваем 'A' и 'C' (как символы)
205 28 00000116 3B0D[39000000]    cmp ecx,[C]
206 29 0000011C 7F0C              jg check_B
207 30 0000011E 8B0D[39000000]    mov ecx,[C]
208 31 00000124 890D[00000000]    mov [max],ecx
209 32                          ; ----- Преобразование 'max(A,C)' из символа в число
210 33                          check_B

```

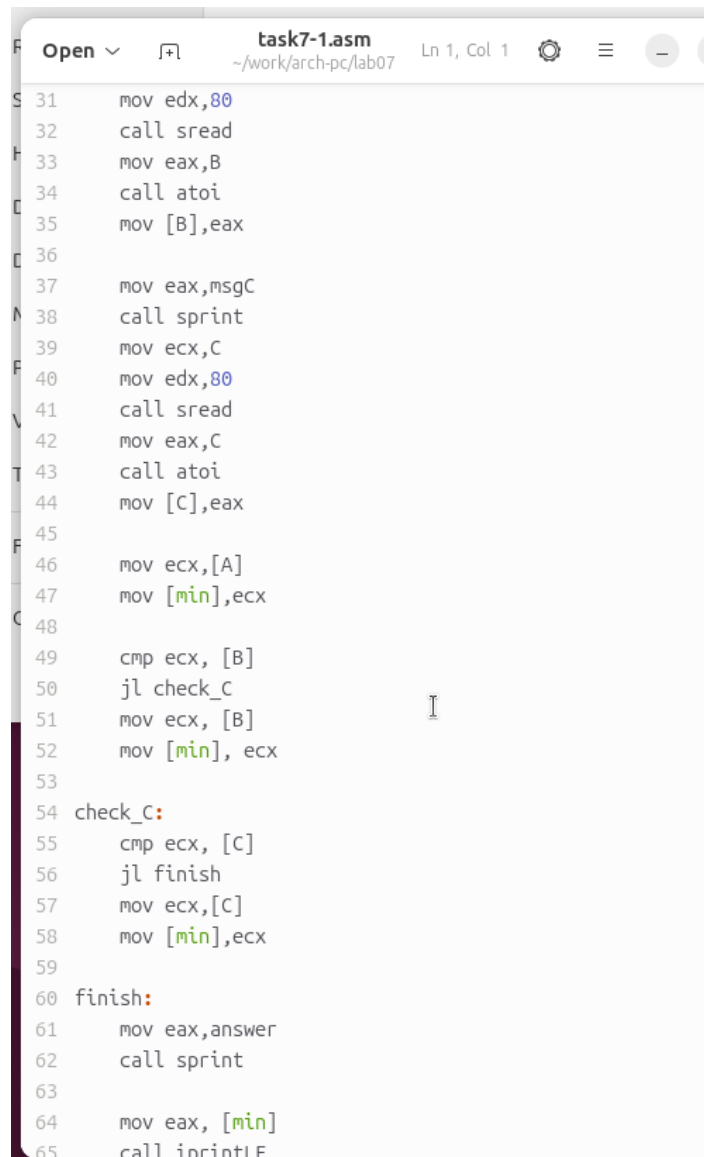
Рисунок 2.11: Файл листинга с ошибкой lab7-2

Объектный файл не смог создаваться из-за ошибки. Но получился листинг, где выделено место ошибки.

2.3 Задание для самостоятельной работы

Напишите программу нахождения наименьшей из 3 целочисленных переменных a, b и c. Значения переменных выбрать из табл. 7.5 в соответствии с вариантом, полученным при выполнении лабораторной работы № 6. Создайте исполняемый файл и проверьте его работу

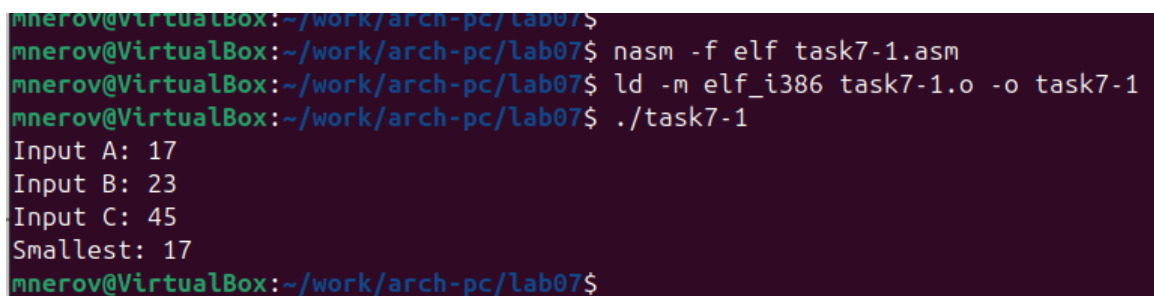
для варианта 1 - 17,23,45



```
task7-1.asm
~/work/arch-pc/lab07
Ln 1, Col 1

31  mov edx,80
32  call sread
33  mov eax,B
34  call atoi
35  mov [B],eax
36
37  mov eax,msgC
38  call sprint
39  mov ecx,C
40  mov edx,80
41  call sread
42  mov eax,C
43  call atoi
44  mov [C],eax
45
46  mov ecx,[A]
47  mov [min],ecx
48
49  cmp ecx, [B]
50  jl check_C
51  mov ecx, [B]
52  mov [min], ecx
53
54  check_C:
55  cmp ecx, [C]
56  jl finish
57  mov ecx,[C]
58  mov [min],ecx
59
60  finish:
61  mov eax,answer
62  call sprint
63
64  mov eax, [min]
65  call iprintf
```

Рисунок 2.12: Программа в файле task7-1.asm



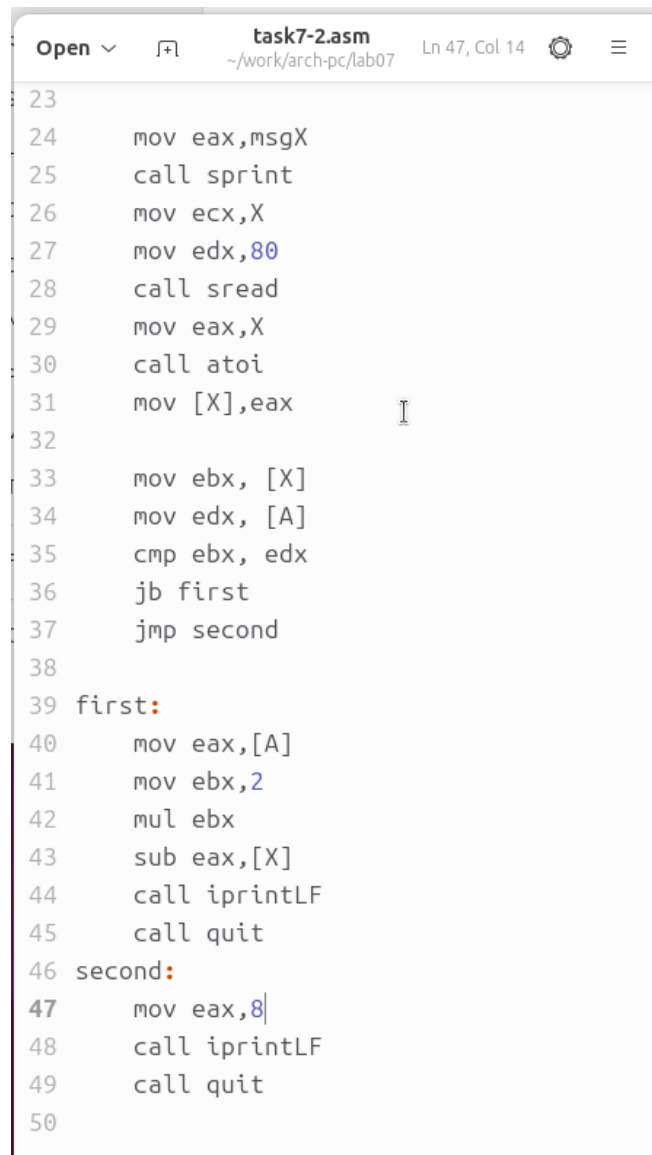
```
mnerov@VirtualBox:~/work/arch-pc/lab07$
mnerov@VirtualBox:~/work/arch-pc/lab07$ nasm -f elf task7-1.asm
mnerov@VirtualBox:~/work/arch-pc/lab07$ ld -m elf_i386 task7-1.o -o task7-1
mnerov@VirtualBox:~/work/arch-pc/lab07$ ./task7-1
Input A: 17
Input B: 23
Input C: 45
Smallest: 17
mnerov@VirtualBox:~/work/arch-pc/lab07$
```

Рисунок 2.13: Запуск программы task7-1.asm

Напишите программу, которая для введенных с клавиатуры значений x и a вычисляет значение заданной функции $f(x)$ и выводит результат вычислений. Вид функции $f(x)$ выбрать из таблицы 7.6 вариантов заданий в соответствии с вариантом, полученным при выполнении лабораторной работы № 7. Создайте исполняемый файл и проверьте его работу для значений X и a из 7.6.

для варианта 1

$$\begin{cases} 2a - x, & x < a \\ 8, & a \geq 0 \end{cases}$$



```
task7-2.asm
~/work/arch-pc/lab07  Ln 47, Col 14

23
24     mov eax,msgX
25     call sprint
26     mov ecx,X
27     mov edx,80
28     call sread
29     mov eax,X
30     call atoi
31     mov [X],eax
32
33     mov ebx, [X]
34     mov edx, [A]
35     cmp ebx, edx
36     jb first
37     jmp second
38
39 first:
40     mov eax,[A]
41     mov ebx,2
42     mul ebx
43     sub eax,[X]
44     call iprintLF
45     call quit
46 second:
47     mov eax,8
48     call iprintLF
49     call quit
50
```

Рисунок 2.14: Программа в файле task7-2.asm

```
mnerov@VirtualBox:~/work/arch-pc/lab07$  
mnerov@VirtualBox:~/work/arch-pc/lab07$ nasm -f elf task7-2.asm  
mnerov@VirtualBox:~/work/arch-pc/lab07$ ld -m elf_i386 task7-2.o -o task7-2  
mnerov@VirtualBox:~/work/arch-pc/lab07$ ./task7-2  
Input A: 2  
Input X: 1  
3  
mnerov@VirtualBox:~/work/arch-pc/lab07$ ./task7-2  
Input A: 1  
Input X: 2  
8  
mnerov@VirtualBox:~/work/arch-pc/lab07$
```

Рисунок 2.15: Запуск программы task7-2.asm

3 Выводы

Изучили команды условного и безусловного переходов, познакомились с фалом листинга.